Auto-évaluation

Projet - Concevez et déployez un système RAG



Un dernier doute avant l'envoi de vos livrables ?

Pour vérifier la qualité de votre travail :

- cochez les cases ci-dessous : elles indiquent que vous avez bien pris en compte chaque indicateur de réussite ;
- renseignez, si besoin, la colonne "Notes" avec des commentaires sur vos livrables / vos étapes. Ils seront des points de discussion avec votre mentor pendant votre session de bilan / soutenance.

Quand toutes les cases de ce document seront cochées, vous pourrez déposer vos livrables sur la plateforme.

Bonne réussite !

Compétences	Livrables	Indicateurs de réussite de l'activité	Notes
Identifier ou créer un modèle d'apprentissage adapté aux contraintes et aux besoins métiers	Rapport technique intégré dans le Github	■ J'ai inclus dans le rapport un schéma UML clair de l'architecture du système. ■ J'ai expliqué dans le rapport le rôle de chaque composant de mon système.	
	Code du système RAG intégré dans le Github	■ J'ai sélectionné un modèle pertinent, adapté aux exigences de performance et de précision. ■ J'ai justifié mes choix dans le rapport technique (modèle, architecture, résultats, limites, améliorations possibles). ■ J'ai adapté le modèle aux spécificités métier (prompts, base de connaissance). ■ J'ai transformé correctement les descriptions d'événements en vecteurs sémantiques via Mistral. ■ J'ai intégré le modèle dans une chaîne LangChain interagissant avec Faiss. ■ J'ai produit des réponses cohérentes et pertinentes à partir du jeu de test annoté.	

Créer les processus de test	Scripts de test unitaires intégrés dans le Github	■ J'ai créé des tests unitaires pour valider les scripts de récupération, nettoyage, vectorisation, interrogation et génération. ■ Mes tests sont relançables automatiquement (via script ou pipeline). ■ J'ai documenté les résultats des tests avec des logs ou un résumé lisible. ■ J'ai utilisé des indicateurs clairs (score de similarité, couverture, taux de réponse correcte). ■ J'ai identifié les erreurs fréquentes et les limites de mon système. ■ J'ai structuré mon dépôt de façon lisible : répertoires séparés pour les scripts, tests, données, API, et documentation. ■ Un évaluateur peut facilement naviguer dans les fichiers, lancer le projet, et tester les fonctionnalités.	
	Jeux de test annotés intégrés dans le Github	■ J'ai conçu un jeu de données annoté pour évaluer la qualité des réponses générées. ■ J'ai utilisé un script de test automatisé pour comparer les réponses obtenues aux réponses attendues.	
Identifier et configurer une API compatible et l'intégrer afin de permettre l'accès aux résultats	API REST (FastAPI ou Flask) intégré dans le Github	■ J'ai mis en place une API REST fonctionnelle avec les endpoints nécessaires (`/ask`, `/rebuild`). ■ J'ai rendu l'API robuste (gestion des erreurs, validation des paramètres, pagination si nécessaire). ■ J'ai fourni un script permettant de lancer l'ensemble localement.	

		■ J’ai nettoyé et préparé les données pour qu’elles soient exploitables par l’index. ■ J’ai documenté clairement les endpoints, les paramètres attendus et les réponses retournées. ■ J’ai testé mon API localement et fourni des exemples via Swagger, Postman ou curl.	
Exposer les résultats aux directions / services métiers (via une API) en vue de leur exploitation	Script de déploiement intégré dans le Github Présentation, Démonstration	■ J’ai conçu une API REST accessible, permettant d’interroger le système RAG et d’obtenir des réponses enrichies. ■ J’ai défini des endpoints adaptés à un usage métier (`/ask`, `/metadata`, `/health`). ■ J’ai fourni des exemples d’utilisation concrets (requêtes types via curl ou Python). ■ J’ai géré les cas limites (pas de réponse, erreurs de requête). ■ J’ai structuré les réponses de manière exploitable par des services externes (JSON clair). ■ Je saurai réaliser une démonstration fluide de mon système, en illustrant avec des cas réalistes. ■ Je saurai expliquer mes choix techniques à des non-techniciens.	